

תמליל כיתה מלא למודול 1 - המפתח המועצם

בבינה מלאכותית

קורס: פיתוח תוכנה מועצם בינה מלאכותית

מודול: 1 - המפתח המועצם: תפיסת עולם ואקוסיסטם

מטרה: תמליל דיבור מלא למרצה בכיתה

סמנים: `module1_slides.pptx`, `module1_companion_notebook.ipynb`,

`module1_demo_runbook.md`, `module1_cheat_sheet.md` ו-`module1_demo_runbook.md`

גרסה: תמליל v1

כיצד להשתמש בתמליל

זהו תסריט דיבור לכיתה, לא תקציר. הוא כתוב בגוף ראשון, כך שהמרצה יכול לקרוא אותו ישירות, להתאים אותו לסגנונו, או להשתמש בו כמדריך הכנה מפורט.

השתמשו במצגת כעמוד השדרה של הקצב. השתמשו במחברת הנלווית כשכבת ההעמקה והעיון. השתמשו ב- `demo_repo/` ו- `module1_demo_runbook.md` כאשר רוצים להפוך את הרעיונות למוחשיים.

חלקים המסומנים **חובה** הם ליבת המודול ויש ללמד אותם גם כאשר הזמן קצר. חלקים המסומנים **רשות** / **ניתן לרפרף אם חסר זמן** אפשר לקצר או לדלג עליהם כאשר המחזור זקוק ליותר זמן לדיון. "רשות" אינו אומר "לא חשוב"; משמעותו שהמשך הקורס עדיין יהיה ברור גם אם החלק הזה יועבר בקצרה.

העברת ליבה מומלצת היא כשעתיים. אם המחזור מנוסה מאוד או שואל הרבה שאלות, המודול יכול להתרחב לשעתיים וחצי או שלוש שעות באמצעות דיון עמוק יותר באקוסיסטם הכלים, בממשל, ובהפעלה הכיתתית של תרגיל הסיווג.

לפני שקף 1 - הכנת המרצה

לפני תחילת השיעור, כדאי לפתוח מראש:

- `module1_slides.pptx`
- `module1_companion_notebook.ipynb`
- `module1_demo_runbook.md`
- `module1_cheat_sheet.md`
- תיקיית `demo_repo/`
- סביבת פיתוח עם `GitHub Copilot`, אם מתוכננת הדגמה חיה

אני רוצה שגם משפט אחד יהיה גלוי לי בהערות המרצה:

בינה מלאכותית יכולה להאיץ עבודת תוכנה, אבל המפתחים עדיין אחראים לספציפיקציה, לארכיטקטורה, לביקורת, לאימות, לאבטחה ולתחזוקה ארוכת טווח.

זה בעצם כל המודול במשפט אחד. כל שאר החומר תומך במשפט הזה.

שקף 1 - כותרת: פיתוח תוכנה מועצם בינה מלאכותית

סמן מצגת: מודול 1 - המפתח המועצם בבינה מלאכותית

סמן מחברת: סעיף 1, יעדי למידה

חובה

ברוכים הבאים לכולם.

במודול הזה אנחנו מניחים את היסודות לכל הקורס. שם המודול הוא **המפתח המועצם בבינה מלאכותית**, והנושאים המרכזיים הם תפיסת עולם, אקוסיסטם, ציפיות וממשל.

אני רוצה להתחיל ממה שהמודול הזה איננו. זה לא שיעור מוטיבציה שמטרתו לשכנע אותנו שבינה מלאכותית היא קסם. זה גם לא שיעור אזהרה שמטרתו לשכנע אותנו שכל כלי בינה מלאכותית הוא מסוכן ולכן אסור להשתמש בו. שתי העמדות האלה פשטניות מדי עבור מפתחים שעובדים בעולם האמיתי.

המציאות המעשית היא כזו: כלי בינה מלאכותית כבר מספיק שימושיים כדי שהתעלמות מהם תהיה בדרך כלל טעות. מצד שני, הם עדיין מספיק לא אמינים כדי שאמון עיוור בהם יהיה גם הוא טעות. לכן השאלה המקצועית איננה "האם בינה מלאכותית טובה או רעה?". השאלה המקצועית היא: "עבור המשימה הזאת, בהקשר הזה, כמה אחריות אני יכול להעביר לכלי בצורה בטוחה, וכיצד אוודא את התוצאה?".

זו תפיסת העולם שאני רוצה שתצאו איתה מהיום.

לאורך הקורס נשתמש בכמה כלים, ובדרך כלל נקודת הייחוס הכיתתית תהיה `GitHub Copilot`, משום שיש לכם גישה אליו. אבל כבר מההתחלה אני רוצה להימנע ממלחמת כלים. המיומנות העמידה לאורך זמן איננה לזכור איפה נמצא כל כפתור במוצר מסוים. המיומנות העמידה היא להבין צורות אינטראקציה, הקשר, הרשאות, ביקורת ובעלות.

לכן, כשאני אומר "מפתח מועצם בבינה מלאכותית", אני לא מתכוון למפתח שנותן לכלי לכתוב הכל במקומו. אני מתכוון למפתח שידוע להשתמש בבינה מלאכותית כמכפיל כוח, תוך שמירה על שיקול דעת הנדסי.

התזה המרכזית מופיעה בשקף:

בינה מלאכותית יכולה להאיץ עבודת תוכנה, אבל המפתחים עדיין אחראים לספציפיקציה, לארכיטקטורה, לביקורת, לאימות, לאבטחה ולתחזוקה ארוכת טווח.

נחזור למשפט הזה שוב ושוב. הוא יהיה חשוב במודול 2, כאשר נשתמש בבינה מלאכותית כחלק מתהליך פיתוח יומיומי. הוא יהיה חשוב במודול 3, כאשר נדבר על Vibe Coding. הוא יהיה חשוב באיכות קוד, באבטחה, באוטומציות עבודה ובמודולים המאחרים יותר.

כדי להפוך את זה למוחשי, נתחיל בשאלה קטנה.

נניח שיש לכם פונקציה שמפרמטת מחרוזת תאריך. האם הייתם מאפשרים לכלי בינה מלאכותית להציע מימוש? רובנו כנראה נגיד שכן.

עכשיו נניח שיש לכם רכיב `middleware` של הזדהות והרשאות בפרודקשן, שמשותף לשלושה שירותים. האם הייתם מאפשרים לכלי לכתוב אותו אוטומטית ולמזג את התוצאה? רובנו נגיד שלא, או לפחות לא בלי גבולות ברורים מאוד.

שתי הדוגמאות הן "בינה מלאכותית בפיתוח תוכנה", אבל פרופיל הסיכון שונה לחלוטין. ההבדל הזה הוא מה שהמודול הזה מלמד אותנו לראות.

שקף 2 - למה הקורס הזה קיים עכשיו

סמן מצגת: למה הקורס קיים עכשיו

סמן מחברת: סעיף 1 - מה השתנה עד 2026

חובה

בואו נדבר רגע על למה הקורס הזה קיים עכשיו, ולמה קורס כזה נראה היום אחרת מכפי שהיה נראה לפני כמה שנים.

בגל הראשון של כלי פיתוח מבוססי בינה מלאכותית, המסגור הנפוץ היה צר יחסית: הבינה המלאכותית עוזרת לי לכתוב קטעי קוד. היא משלימה שורה, אולי פונקציה, אולי קוד חזרתי. זה כבר היה שימושי, אבל המודל המנטלי היה בעיקר `autocomplete`.

עד 2025 ו-2026, המסגור המעשי השתנה. בינה מלאכותית כבר לא רק מציעה שורה. היא יכולה לבדוק מאגר קוד, להסביר קבצים, להציע שינויים, לערוך כמה קבצים, ליצור בדיקות, להריץ פקודות אם מאפשרים לה, לסכם `pull request`, ולעיתים לעבוד מתוך משימה או `issue` ולהחזיר ענף, `PR` או דוח.

לכן השקף מציג מעבר:

- קודם: "בינה מלאכותית עוזרת לי לכתוב קוד".
- אחר כך: "בינה מלאכותית עוזרת לי לבנות פיצ'ר".
- עכשיו: "בינה מלאכותית יכולה להריץ חלקים מתהליך עבודה".

המעבר הזה משנה את מה שמפתחים צריכים ללמוד.

אם בינה מלאכותית רק משלימה שורה, המיומנות המרכזית היא להחליט אם השורה הזאת מתקבלת. אבל אם הכלי יכול לערוך כמה קבצים, להריץ בדיקות או לעבוד מול מערכות חיצוניות, המיומנות המרכזית הופכת להיות תכנון תהליך עבודה בטוח סביב הכלי.

לכן צוואר הבקבוק אינו רק מהירות הקלדה. צוואר הבקבוק הופך להיות ספציפיקציה, הקשר, ביקורת, אימות והאצלה בטוחה.

דוגמה פשוטה: נניח שאני מבקש מכלי בינה מלאכותית:

Add pagination to the API.

אם זה היה רק `autocomplete`, אולי הייתי מקבל כמה שורות בתוך `endpoint` אחד. אבל עוזר שמכיר את המאגר עשוי לשנות את ה-`endpoint`, לשנות את סכמת התשובה, להוסיף בדיקות, לעדכן תיעוד ואולי להוסיף פונקציית עזר. זה יכול להיות מועיל מאוד. זה גם יכול לשבור תאימות לאחור אם ה-API החזיר בעבר רשימה ועכשיו מחזיר אובייקט עם מטא-דאטה.

הבעיה איננה האם הכלי מסוגל. הבעיה היא האם המשימה הוגדרה בצורה ברורה מספיק, האם לכלי היה ההקשר הנכון, האם ה-`diff` ניתן לביקורת, והאם אנחנו יודעים איך לוודא את התוצאה. זו הסיבה שהמודול הראשון מתחיל בתפיסת עולם ובאקוסיסטם, לפני שאנחנו קופצים לכלים עצמם.

שקף 3 - מה המשמעות של פיתוח מועצם בבינה מלאכותית

סמן מצגת: הגדרה

סמן מחברת: סעיף 2 - הגדרה

חובה

עכשיו נגדיר את המונח בזהירות.

פיתוח תוכנה מועצם בבינה מלאכותית הוא שימוש מכוון במערכות בינה מלאכותית כדי לסייע, להאיץ או לאפשר אוטומציה חלקית של עבודת תוכנה לאורך מחזור חיי הפיתוח, תוך השארת האחריות לנכונות, בטיחות, תחזוקתיות והתאמה מוצרית בידי בני אדם.

להגדרה הזאת יש שני חלקים, ושניהם חשובים.

החלק הראשון אומר שבינה מלאכותית יכולה לעזור לאורך מחזור חיי הפיתוח, לא רק בזמן כתיבת קוד. היא יכולה לעזור בהבנת דרישות, בהיכרות עם קוד קיים, ביצירת בדיקות, בהסבר שגיאות, ביצירת תיעוד, בביקורת על שינויים, בסיכום `issues`, בהכנת הערות גרסה ולעיתים גם בתזמור תהליכי עבודה.

החלק השני אומר שהאחריות נשארת אצל בני אדם. זו לא הערה משפטית שולית. זה מרכז הקורס.

אם שינוי שנוצר על ידי בינה מלאכותית מכניס פרצת אבטחה, ההסבר "ה-AI כתב את זה" לא יעזור לצוות. אם חבילת בדיקות שנוצרה אוטומטית מפספסת את מקרי הקצה החשובים, מספר הבדיקות לא מגן עלינו. אם כלי מסביר קוד בביטחון אבל טועה, ואנחנו מעבירים את ההסבר לצוות אחר, זו עדיין טעות שלנו.

לכן משפט העבודה שלנו הוא:

בינה מלאכותית משנה את המכניקה של עבודת תוכנה, לא את הצורך בשיקול דעת הנדסי.

חשוב לכוון ציפיות. אני לא רוצה שתהפכו לאיטיים יותר מפני שאתם חוששים מכל הצעת AI. אני גם לא רוצה שתהפכו לפזיזים רק משום שטייטה ראשונה נראית מרשימה. אני רוצה שתהפכו מהירים יותר היכן שבטוח להאיץ, וממושמים יותר היכן שהסיכון גבוה.

בפועל, לפעמים השימוש הטוב ביותר בבינה מלאכותית הוא יצירת קוד. לפעמים השימוש הטוב ביותר הוא לבקש תוכנית. לפעמים כדאי לשאול: "מה אני מפספס?". לפעמים השימוש הטוב ביותר הוא הסבר במצב קריאה בלבד. ולפעמים נכון לא להשתמש בבינה מלאכותית בחלק הרגיש עצמו, אלא רק סביבו.

לדוגמה, בפיצ'ר אבטחה או הרשאות, ייתכן שלא אבקש מה-AI לממש את מודל האבטחה. אבל כן אבקש ממנו לנסח רשימת שאלות תכנון, ליצור בדיקות אחרי שהגדרתי את ההתנהגות, או לבדוק האם הודעות שגיה עלולות לחשוף מידע.

זה ההבדל בין שימוש בבינה מלאכותית ככלי לבין שימוש בבינה מלאכותית כתירוץ.

שקף 4 - מ-Autocomplete לעבודה מואצלת

סמן מצגת: מ-autocomplete לעבודה מואצלת

סמן מחברת: סעיף 3 - ספקטרום האינטראקציה

חובה

השקף הזה מציג רצף: להציע, להסביר, לערוך, להריץ, לפתוח PR, ולפעול באמצעות כלים.

אלה לא רק יכולות מוצר. אלה רמות שונות של האצלת אחריות.

בצד שמאל, העוזר מציע טקסט. זה מוכר לנו: השלמת קוד, גוף פונקציה, אולי שינוי קטן. רדיוס הפגיעה בדרך כלל מקומי, ועומס הביקורת נמוך יחסית.

אחר כך העוזר מסביר. זה שימוש חשוב שלעיתים מזלזלים בו. בקוד לא מוכר, הסבר יכול להיות חשוב יותר מיצירת קוד. הוא יכול לעזור לי להתמצא לפני שאני משנה משהו.

אחר כך העוזר עורך. עכשיו הוא משנה קבצים. הביקורת הופכת חשובה יותר, כי הכלי יכול לבצע שינויים שלא דמיינתי במפורש.

אחר כך הוא מריץ פקודות: בדיקות, linters, בנייה, או סקריפטים אחרים. זה שימושי, אבל עכשיו יש חשיבות להרשאות ולבטיחות הפקודות.

אחר כך הוא עשוי לפתוח PR או לעבוד על ענף. זה נשמע כמו תוצר צוותי רגיל, אבל עכשיו תהליך הביקורת האנושי הופך לשער המרכזי.

לבסוף, עם כלים מחוברים, העוזר עשוי לפעול על פני כמה מערכות: מאגר קוד, משימות, תיעוד, CI, מסדי נתונים, כלי פריסה או APIs פנימיים. זו כבר לא רק כתיבת קוד. זה תכנון תהליך עבודה.

המשפט החשוב הוא:

רק משום שכלי יכול להריץ פקודה, לא אומר שהוא צריך להריץ כל פקודה. רק משום שכלי יכול לגעת בכמה קבצים, לא אומר שהמשימה צריכה להיות רחבה. רק משום שסוכן ענן יכול לעבוד מתוך `issue`, לא אומר שה- `issue` הוגדר טוב מספיק.

מודל מנטלי שימושי: ככל שאנחנו זזים שמאלה לימין, בדרך כלל גדלים שלושה דברים:

1. רמת האוטונומיה;

2. גישת ההקשר;

3. עומס הביקורת.

לכן צריך סיבה טובה לזוז ימינה. לא משתמשים במצב האוטונומי ביותר רק כי הוא קיים. זזים ימינה כאשר המשימה ברורה, תחומה וניתנת לאימות.

במודול 2 נתרגל את זה במשימות פיתוח יומיומיות. במודול 3, כשנדבר על `Vibe Coding`, זה יהיה חשוב אפילו יותר, כי הטיטה הראשונה יכולה להיות גדולה ומהירה.

שקף 5 - ספקטרום האינטראקציה

סמן מצגת: עוזר, צ'אט, מתכנת זוגי, סוכן IDE, סוכן קידוד, תהליך עבודה

סמן מחברת: סעיף 3 - ספקטרום האינטראקציה

חובה

נגדיר את צורות האינטראקציה בצורה מדויקת יותר.

ראשית: **עוזר Inline**. זה מצב ה- `autocomplete` הקלאסי או עריכה מקומית קטנה. הוא טוב ל- `boilerplate`, לדפוסים צפויים ולקוד חזרתי. כשל נפוץ הוא שההצעה נכונה מקומית אבל מפספסת הקשר רחב יותר.

שנית: **עוזר צ'אט**. זה שימושי להסבר קוד, השוואת חלופות, סיכום שגיאות, ניסוח תיעוד או שאלות מושגיות. לעיתים זה בטוח יותר כי אפשר לשמור אותו במצב קריאה בלבד. אבל כשל נפוץ הוא הסבר בטוח בעצמו שאינו באמת מעוגן בקוד.

שלישית: **מתכנת זוגי מבוסס AI**. כאן העוזר עוזר לדבג, לתכנן בדיקות, לבצע ריפקטורינג או לחשוב דרך שינוי. אתם עדיין מכוונים אותו באופן רציף. זה בדרך כלל מצב העבודה היומיומי החשוב ביותר.

רביעית: **סוכן IDE**. עכשיו הכלי יכול לבצע שינויים תחומים בכמה קבצים ואולי להריץ פקודות. זה יכול להיות מאוד יעיל עבור משימה ברורה וניתנת לבדיקה. הסיכון הוא עריכת יתר: שינוי יותר מדי קבצים, ריפקטורינג לא קשור, או סיפוק הבדיקות בצורה שטחית.

חמישית: **סוכן קידוד או סוכן ענן**. זו עבודה שמזכירה יותר `issue`: נותנים משימה, והסוכן עובד בסביבה ומחזיר ענף, `PR` או דוח. זה יכול להיות שימושי למשימות מוגדרות היטב, אבל עומס הביקורת גבוה.

שישית: **תהליך עבודה סוכני**. כאן הבינה המלאכותית מחברת כמה מערכות: מאגר קוד, מערכת משימות, תיעוד, CI, תמיכה, כלי תצפית וכדומה. זו כבר לא רק כתיבת קוד. זה תכנון תהליכי עבודה.

חשוב להבין: אלה לא קטגוריות מוסריות. אני לא אומר שהצד השמאלי טוב והימני רע. אני אומר שכל מצב דורש אסטרטגיית ביקורת אחרת.

אם עוזר `inline` מציע לשנות שם של משתנה מקומי, אני בודק את ה-`diff` המקומי.

אם סוכן IDE משנה שישה קבצים, אני צריך לבדוק התנהגות, בדיקות, הנחות ותופעות לוואי.

אם סוכן תהליכי עבודה מתחבר למשימות, תיעוד ו-CI, אני צריך לחשוב על הרשאות, לוגים, אישורים ויכולת חזרה לאחור.

לכן חשוב לי שתדברו במדויק. במקום לומר "השתמשתי ב-AI", אמרו למשל:

השתמשתי בעוזר במצב קריאה בלבד כדי למפות את המודול.

או:

השתמשתי בסוכן IDE לשינוי אימות תחום בשני קבצים ובדקתי את ה-`diff`.

או:

השתמשתי ב-AI כדי ליצור טיוטת בדיקות ואז ערכתי ידנית את ה-`assertions`.

שפה כזאת הופכת בעלות לגלויה.

שקף 6 - Copilot ככלי הבסיס הכיתתי

סמן מצגת: Copilot כבסיס כיתתי

סמן מחברת: סעיף 4 - אקוסיסטם הכלים נכון ליוני 2026
חובה

בקורס הזה, `GitHub Copilot` הוא נקודת הייחוס הכיתתית המשותפת שלנו. הסיבה המעשית פשוטה: יש לכם גישה אליו, והוא מכסה מספיק צורות עבודה כדי ללמד את הדפוסים המרכזיים.

נוכל להשתמש בו כהשלמה מקומית. נוכל להשתמש בצ'אט בתוך סביבת הפיתוח. בהתאם לחשבון ולסביבה, נוכל גם לדבר על עריכה סוכנית, עזרה בביקורת קוד, הנחיות מאגר, קבצי פרומפט ותהליכי עבודה סביב `GitHub`.

אבל המטרה איננה "ללמוד כפתורים של Copilot". ממשקי מוצר משתנים. שמות פיצ'רים משתנים. זמינות תלויה בחשבון, במדיניות הארגון ובגרסת ה-IDE. המטרה היא ללמוד הרגלי פיתוח ממושמעים בעזרת בינה מלאכותית, על בסיס כלי משותף.

לכן, כשאנחנו מדברים על Copilot, אנחנו בדרך כלל נשאל:

- איזה הקשר הוא יכול לראות?
- מה הוא יכול לשנות?
- האם הוא יכול להריץ פקודות?
- איך מגבילים אותו?
- איזה תוצר אנחנו בודקים?
- מה עדיין בבעלות האדם?

השאלות האלה יעברו גם ל-Claude Code, ל-Codex, לכלי Gemini או Google, ל-Continue, ל-OpenHands ולכלים עתידיים שעוד לא ראינו.

אם אתם כבר משתמשים בכלי אחר, זה בסדר גמור. הניסיון שלכם אפילו יכול להעשיר את הדיון. אבל לצורך מעבדות, הדגמות ושפה משותפת, נעגן בדרך כלל ב-Copilot.

זה גם מקום טוב להזכיר מדיניות ארגונית. יש ארגונים שמאשרים כלים מסוימים ולא אחרים. יש ארגונים שמאפשרים AI על קוד ציבורי אבל לא על קוד קנייני. יש ארגונים שמאפשרים רק עוזרים מנוהלים ארגונית ולא חשבונות אישיים. התשובה הנכונה בצוות אמיתי איננה "נשתמש במה שנוח". התשובה הנכונה היא להבין את גבולות הכלים המאושרים.

בכיתה נשתמש במאגרים לדוגמה ובנתונים סינתטיים כדי להתמקד בתהליך העבודה ולא בשאלות חשיפה משפטיות או עסקיות.

שקף 7 - אקוסיסטם הכלים נכון ליוני 2026

סמן מצגת: אקוסיסטם נוכחי

סמן מחברת: סעיף 4 - אקוסיסטם הכלים

רשות / ניתן לרפרף אם חסר זמן

השקף הזה איננו דירוג מוצרים. זו תמונת נוף.

GitHub Copilot הוא ברירת המחדל הכיתתית שלנו. הוא שימושי כי הוא מכסה השלמות, צ'אט, הקשר מאגר, עזרה בביקורת ותהליכים שהופכים יותר ויותר סוכניים.

Claude Code הוא דוגמה חזקה לסוכן קידוד שמכיר מאגר, במיוחד לעבודה טרמינלית או משולבת IDE. הוא שימושי כשמדברים על הרשאות, זיכרון פרויקט ושינויים מרובי קבצים.

OpenAI Codex הוא דוגמה נוספת לתהליך קידוד סוכני דרך CLI, IDE, אפליקציה וסביבות ענן. הוא שימושי כשמדברים על משימות מואצלות, עבודה מקבילה ותוצרים ניתנים לביקורת.

האקוסיסטם של Google השתנה לאורך הזמן, במיוחד סביב **Gemini Code Assist**, **Gemini CLI** ו-**Antigravity**. גבולות המוצר המדויקים עשויים להשתנות, ולכן לא כדאי ללמד לעומק ממשק אחד. הנקודה העמידה היא שגם Google היא חלק מהמעבר הרחב לכלי פיתוח סיעניים וסוכניים.

Continue - OpenHands שימושיים כדוגמאות פתוחות או מנוהלות עצמית. הם עוזרים לדבר על בחירת מודל, שליטה מקומית ועלות תפעולית.

MCP, או Model Context Protocol, אינו עוזר קידוד בפני עצמו. הוא מושג אקוסיסטם: דרך לחבר אפליקציות AI לכלים ולמקורות נתונים. היום רק נציג אותו בקצרה ונחזור אליו בהמשך.

אם הזמן קצר, אל תתעכבו כאן. הנקודה היא:

שמות מוצרים משתנים מהר יותר מאחריות הנדסית.

לכן נסו לסווג כלים לפי צורת העבודה:

- עוזר;
- צ'אט;
- מתכנת זוגי;
- סוכן IDE;
- סוכן issue או ענן;
- תהליך עבודה מחובר לכלים.

הסיווג הזה ימשיך להיות שימושי גם כאשר האקוסיסטם ישתנה.

שקף 8 - עוזר מול מתכנת זוגי מול סוכן

סמן מצגת: אוצר מילים: עוזר, מתכנת זוגי, סוכן

סמן מחברת: סעיף 3 וסעיף 6 - סולם הרשאות חובה

נחدد שלושה מונחים: עוזר, מתכנת זוגי וסוכן.

עוזר מציע. אתם מחליטים מה הצעד הבא. הביקורת בדרך כלל מקומית. זה המצב שרבים חווים לראשונה דרך `autocomplete`.

מתכנת זוגי משתף פעולה. אתם עדיין מובילים, אבל העוזר יכול לחשוב, לנסח, להסביר, להציע בדיקות ולהגיב לתיקונים שלכם. זה בדרך כלל מצב העבודה היומיומי החשוב ביותר.

סוכן פועל. הוא עשוי לבדוק קבצים, לערוך קבצים, להריץ פקודות, ליצור תוצרים או לעבוד מול מערכות חיצוניות. זה לא אומר שהוא אוטונומי במובן האנושי. זה אומר שיש לו שטח פעולה רחב יותר.

קיצור הדרך הגרוע הוא:

ה-AI עשה את זה.

המשפט הזה מסתיר אחריות. הוא מסתיר את גבולות המשימה, את הביקורת, את הבדיקות ואת ההחלטה האנושית.

האצלתי משימה תחומה, בדקתי את ה-diff, הרצתי את הבדיקות האלה וקיבלתי את הסיכון הזה.

זה הניסוח המקצועי.

עכשיו נחבר את זה להרשאות. כלי שרק מציע טקסט מוגבל ברדיוס הפגיעה שלו. כלי שעורך קובץ מסוכן יותר. כלי שמריץ פקודות מסוכן יותר. כלי שיכול לפעול מול מערכות חיצוניות מסוכן עוד יותר.

לכן בכל פעם שמישהו אומר "סוכן", אני רוצה שתשאלו:

- מה הוא יכול לקרוא?
- מה הוא יכול לכתוב?
- מה הוא יכול להריץ?
- מה הוא יכול לשלוח החוצה?
- איפה שער האישור?
- איפה תיעוד הפעולות?

זה יהיה חשוב במיוחד במודולים מאוחרים יותר, כשנדבר על אבטחה, CI/CD, אוטומציות עבודה וכלים מחוברים.

שקף 9 - מודל החלטה של חמש שאלות

סמן מצגת: מודל החלטה

סמן מחברת: סעיף 5 - מודל חמש השאלות
חובה

זה אחד השקפים החשובים במודול.

לפני שמשתמשים בבינה מלאכותית במשימת פיתוח, שאלו חמש שאלות.

ראשית: עד כמה המשימה ברורה?

אם המשימה מעורפלת, AI ימלא את הפערים. לפעמים זה מועיל לסייעור מוחות. לפעמים זה מסוכן כי הכלי ממציא דרישות.

שנית: כמה הקשר נדרש?

פונקציה טהורה עם קלט ופלט ברורים דורשת מעט הקשר. שינוי התנהגות חוצה שירותים דורש הרבה הקשר: ארכיטקטורה, חוזים, נתונים, בדיקות, חוקים מוצריים, הנחות פריסה ונורמות צוות.

שלישית: מה רדיוס הפגיעה אם זה שגוי?

תווית כפתור שגויה איננה כמו לוגיקת הרשאות שגויה. דוגמת README גרועה איננה כמו מיגרציית מסד נתונים גרועה. ככל שרדיוס הפגיעה גדול יותר, כך העבודה צריכה להיות יותר מונחית אדם.

רביעית: האם ניתן לאמת את התוצאה בזול?

זו לעיתים המגבלה הסמויה. אם אני יכול להריץ חבילת בדיקות ממוקדת ולבדוק `diff` קטן, האצלה ל-AI בטוחה יותר. אם האימות דורש שבועות של תצפית בפרודקשן או מומחיות דומיינית עמוקה, ההאצלה צריכה להיות מוגבלת.

חמישית: האם אני צריך הצעה, או שאני צריך שהכלי יפעל?

לפעמים הסבר מספיק. לפעמים תוכנית מספיקה. לפעמים רשימת בדיקה מספיקה. לא תמיד צריך שה-AI יערוך קבצים.

כלל האצבע הוא:

הגדילו אוטונומיה רק כאשר בהירות ואימות משתפרים מהר יותר מהסיכון.

ניישם את זה במהירות.

משימה: יצירת `docstrings` למודול שירות פנימי קטן. המשימה ברורה, רדיוס הפגיעה נמוך, והביקורת קלה. עוזר `inline` או צ'אט הם בחירה טובה.

משימה: מימוש `role-based access control` לשירות פרודקשן. המשימה אולי נשמעת ברורה, אבל ההקשר ורדיוס הפגיעה עצומים. השתמשו ב-AI בזהירות סביב הקצוות: שאלות תכנון, רשימת בדיקה, בדיקות, עזרה בביקורת. אל תאצילו את המימוש בצורה עיוורת.

משימה: הוספת אימות ל-parser קטן ומבודד עם בדיקות. ברור, תחום וקל לאימות. מתכנת זוגי או סוכן IDE מוגבל יכולים להתאים.

מודל ההחלטה הזה איננו תיאורטי. הוא הדרך לבחור צורת עבודה מתאימה לפני כתיבת הפרומפט.

שקף 10 - דוגמאות התאמת כלי למשימה

סמן מצגת: דוגמאות התאמת כלי

סמן מחברת: טבלת דוגמאות בסעיף 5

חובה

נעבור על כמה דוגמאות.

שינוי שם של משתנה מקומי הוא משימה טובה לעוזר. היא מקומית, בסיכון נמוך וקלה לביקורת.

הסבר של parser ישן הוא משימה טובה לצ'אט או למתכנת זוגי. אפשר לעשות אותה במצב קריאה בלבד, והיא עוזרת להתמצא לפני נגיעה בקוד.

יצירת בדיקות לפונקציה טהורה היא משימה טובה למתכנת זוגי. אבל חשוב להדגיש: בדיקות שנוצרו הן טיטוט. הכלי עשוי ליצור בדיקות שעוברות אבל לא מוכיחות את ההתנהגות שחשובה לכם.

מימוש issue קטן ומבודד יכול להתאים לסוכן IDE אם השינוי תחום ויש בדיקות.

שינוי הרשאות, חיוב, מיגרציות נתונים, סודות או לוגיקת פריסה לפרודקשן צריך להישאר מונחה אדם. AI יכול לעזור, אבל תהליך העבודה האנושי חייב להיות חזק יותר.

סיכום שרשור באג הוא משימת צ'אט טובה, כל עוד מטפלים נכון בפרטים רגישים ובודקים את הסיכום. משימת מאגר רב-שלבית יכולה להתאים לסוכן קידוד אם ה-issue ברור והתוצר הוא `diff` או `PR` שניתן לביקורת.

שימו לב שהתשובה אינה "AI כן" או "AI לא". התשובה היא צורת עבודה, עם מגבלות ואימות. למשל:

כן, השתמשו ב-AI כדי ליצור טיוטה ראשונה של בדיקות ל-parser, אבל בדקו ידנית את ה-assertions והוסיפו מקרי קצה.

או:

כן, בקשו מה-AI לנתח middleware של הרשאות, אבל שמרו אותו במצב קריאה בלבד עד שהתכנון והציפיות האבטחתיות ברורים.

או:

כן, השתמשו בסוכן למימוש פילטר UI קטן, אבל הגבילו קבצים, דרשו בדיקות ובדקו את ה-diff.

זו השפה שנשתמש בה לאורך הקורס.

שקף 11 - איפה כלי AI חזקים בדרך כלל

סמן מצגת: חוזקות AI

סמן מחברת: סעיף 7 - השלמה סבירה אך חלקית

חובה

עכשיו נהיה הוגנים כלפי הכלים. כלי קידוד מבוססי AI שימושיים. לעיתים הם שימושיים מאוד.

הם טובים ביצירת שלד ו-`boilerplate`. אם צריך `route` בסיסי ב-FastAPI, שלד של קומפוננט React, `fixture` של `pytest` או `dataclass` פשוט - AI יכול לייצר טיוטה טובה במהירות.

הם טובים בהסבר קוד לא מוכר. גם אם ההסבר לא מושלם, הוא יכול לתת מפה ראשונית.

הם טובים בטרנספורמציות חוזרות: שינוי דפוסים, המרת מבנה אחד לאחר, הוספת אימות דומה בכמה מקרים. אלה משימות טובות לעזור.

הם טובים ביצירת טיוטות בדיקות ותיעוד. שוב: טיוטות, לא הוכחה.

הם טובים בדפוס ריפקטורנינג מוכרים: חילוץ פונקציית עזר, פישוט ענפים כפולים, העברת קוד חוזר לפונקציה.

הם טובים בהתמצאות וחיפוש במאגר, כאשר יש להם גישה לקוד.

הם טובים בעבודה מובנית תחת מגבלות מפורשות.

ואולי הכי חשוב: הם טובים ביצירת טיוטה ראשונה שימושית במהירות.

לכן מטרת הקורס איננה לגרום לכם לחשוד בכל פלט של AI. זה יהיה בזבוז. המטרה היא להשתמש בחוזקות האלה במקום המתאים, ואז להפעיל את משמעת הביקורת הנכונה.

טיוטה ראשונה שנוצרה אוטומטית יכולה להיות מכפיל כוח. אבל טיוטה ראשונה היא עדיין טיוטה ראשונה.

אני אוהב את הביטוי:

טיוטה מהירה, בעלות איטית.

אפשר לקבל משהו על המסך מהר מאוד. עדיין צריך עבודת הנדסה זהירה כדי להפוך אותו לקוד שהצוות באמת מחזיק בבעלותו.

שקף 12 - השלמה סבירה אך חלקית

סמן מצגת: כשל סמוי

סמן מחברת: סעיף 7 - Plausible incompleteness

חובה

זה כנראה שקף הסיכון החשוב ביותר במודול 1.

הפלט המסוכן ביותר של AI לעיתים אינו שטות מובהקת. אם הפלט היה מופרך לחלוטין, היינו דוחים אותו. הפלט המסוכן הוא סביר, מלוטש וחלקי.

הוא מטפל ב-happy path אבל לא במקרה הקצה.

הוא מוסיף בדיקות, אבל הבדיקות רק בודקות `status code`.

הוא מסביר קוד בביטחון, אבל ממציא התנהגות שלא קיימת בקוד.

הוא משתמש בברירת מחדל לא בטוחה, אבל הכל נראה עובד.

הוא משנה יותר קבצים מהנדרש, אבל ה-diff נראה פרודוקטיבי.

הוא כותב תיעוד שנשמע סמכותי, אבל משמיט מגבלות תפעוליות.

לכן ביקורת צריכה להתמקד במה שחסר, לא רק במה שקיים.

דוגמה קטנה: נניח ש-AI יוצר פונקציית הרשמת משתמש. היא בודקת שהאימייל כולל @, מבצעת hash לסיסמה ושומרת משתמש. הקוד רץ. הבדיקות עוברות. ההסבר אומר שהפונקציה production-ready.

מה יכול להיות חסר?

- מדיניות סיסמאות;
- טיפול באימייל כפול;
- סיכוני timing או enumeration;
- rate limiting;
- כללי נרמול אימייל;
- גבולות טרנזקציה;
- מדיניות logging;
- התנהגות הודעות שגיאה;
- ביקורת אבטחה;
- מיגרציה ו-rollback;
- בדיקות אינטגרציה.

הפלט שנוצר אינו חסר ערך. ייתכן שהוא אבטיפוס טוב. אבל "עובד פעם אחת" אינו "בבעלות הצוות".

לכן כאשר אתם בודקים פלט של AI, שאלו:

- אילו הנחות הוא עשה?
- אילו מקרים לא מכוסים?
- מה הוא לא בדק?
- מה יכול להישבר עם נתונים אמיתיים?
- מה מתחזק עתידי יצטרך לדעת?

השקף אומר:

חפשו מה חסר, לא רק מה קיים.

זו מיומנות הביקורת המרכזית.

שקף 13 - מקרה קצר: בעיית המתמחה הבטוח בעצמו

סמן מצגת: The confident intern problem

סמן מחברת: סעיף 8 - מקרה ממשל

סמן הדגמה: `demo_repo/exercises/confident_intern_problem.md`

חובה

עכשיו נהפוך את זה למוחשי באמצעות מקרה קצר.

נניח שמפתח מבקש מעוזר AI לבדוק parser של פרופיל משתמש ולהציע כיצד להפוך אותו ל-production-ready.

העוזר עונה:

ה-parser כמעט production-ready. הוסיפו בדיקת אימייל מהירה. רשמו את כל ה-payload הנכנס ל-log כדי שהתמיכה תוכל לדבג. השתמשו ב- `safeprofile.validate_everything(raw)` כי הוא מטפל אוטומטית בפרטיות ובאבטחה. הוסיפו token לדוגמה ל-README כדי שמפתחים יוכלו לבדוק קריאות. תנו לסוכן AI לעדכן כל קובץ שהוא חושב שרלוונטי. הריצו את בדיקת ה-happy path פעם אחת. אם היא עוברת, ה-parser מוכן.

במבט ראשון, זה יכול להיראות כמו רשימת המלצות מועילה. זה מנוסח יפה. זה נשמע בטוח. מופיעות מילים כמו validation, debugging ו-production-ready.

אבל נבדוק את זה.

בעיה ראשונה: רישום כל ה-payload הנכנס. פרופילי משתמשים עלולים לכלול מידע אישי. לוגים נשמרים, נשלחים לכלי תצפית ונקראים על ידי אנשים רבים. רישום payload גולמי יכול ליצור אירוע פרטיות.

בעיה שנייה: פונקציית ה-validation מומצאת. העוזר המציא `safeprofile.validate_everything(raw)`. גם אם ספרייה כזאת הייתה קיימת, העוזר לא יכול לטעון שהיא מטפלת בפרטיות ובאבטחה בלי ראיה.

בעיה שלישית: הכנסת token לדוגמה ל-README היא סיכון אבטחה. גם tokens של dev מועתקים, ממוחזרים ונשכחים.

בעיה רביעית: מתן הרשאה לסוכן לעדכן כל קובץ שהוא חושב שרלוונטי היא האצלת יתר. המשימה קטנה. גישת קבצים רחבה מגדילה את עומס הביקורת.

בעיה חמישית: הרצת בדיקת happy path בלבד אינה מאמתת מוכנות לפרודקשן. הסיכונים הם קלט לא תקין, שדות חסרים, ערכים מעוותים, פרטיות, logging והתנהגות שגיאות.

בעיה שישית: המסקנה חזקה מדי. "production-ready" לא מוצדק.

לכן אני קורא לזה בעיית המתמחה הבטוח בעצמו. דמיינו מתמחה מהיר מאוד, בטוח מאוד בעצמו, שכותב הסברים מלוטשים אבל עדיין לא מבין את אילוצי הפרודקשן שלכם. אדם כזה יכול להיות מועיל, אבל לא הייתם מקבלים הכל בלי ביקורת.

פלט AI ראוי לאותה סקפטיות מקצועית.

אם אני מריץ את זה כתרגיל בכיתה, אבקש מכם לסמן בעיות לפי קטגוריות:

- פרטיות ודליפת נתונים;
- התנהגות מומצאת;
- סיכון אבטחה;
- אימות חסר;
- סיכון תחזוקה;

שקף 14 - פרטיות, קניין רוחני וממשל

סמן מצגת: פרטיות, IP וממשל

סמן מחברת: סעיף 9 - טיפול במידע

חובה

עכשיו נדבר על גבולות מידע.

יש דברים שבדרך כלל בטוחים לשיתוף בכיתה או בסביבת AI מאושרת: קוד צעצוע, דוגמאות סינתטיות, תיעוד ציבורי, סיכומים פנימיים מאושרים ו-stack traces שעברו הסרה של מידע רגיש.

יש דברים שמצריכים זהירות: קוד קנייני, פרטי ארכיטקטורה, לוגים, stack traces, ממצאי אבטחה ותוכניות מוצר לא ציבוריות. האם מותר לשתף אותם תלוי במדיניות הארגון, בתצורת הכלי ובסיווג המידע.

ויש דברים שלא מדביקים באופן אגבי לכלי AI:

- secrets;
- tokens;
- keys;
- credentials;
- נתוני לקוחות חיים;
- מידע אישי מפוקח;
- dumps של מסדי נתונים מפרודקשן;
- payloads רגישים של לקוחות.

רוב המפתחים יודעים לא להדביק API key. אבל דליפות קורות בדרכים פחות ברורות.

Stack trace יכול לכלול אימייל של לקוח.

שורת log יכולה לכלול authorization header.

צילום מסך יכול לכלול URL פנימי או token.

קובץ קונפיגורציה יכול לכלול secrets בהערות.

דוח באג יכול לכלול payload אמיתי של לקוח.

שורת מסד נתונים שנראית כמו דוגמה יכולה לכלול מידע אישי.

לכן לפני שימוש ב-AI שאלו:

- האם אפשר לשחזר את הבעיה עם נתונים סינתטיים?

- האם אפשר לצמצם את ההקשר לדפוס הטכני המינימלי?
- האם אפשר להסיר מזהים בלי לאבד משמעות?
- האם הארגון מאפשר שימוש בכלי הזה עבור סיווג המידע הזה?
- האם החשבון מנוהל ארגונית או אישי?

בקורס הזה נשתמש בפרויקטי צעצוע ובדוגמאות סינתטיות. זה מכון. כך אפשר לתרגל תהליכי עבודה בלי לחשוף קוד או מידע אמיתי.

ממשל אינו ההפך מפרודוקטיביות. ממשל טוב הוא מה שהופך AI לשימושי בצוותים אמיתיים.

שקף 15 - תצוגה מקדימה: כלים מחוברים ו-MCP

סמן מצגת: תצוגת MCP

סמן מחברת: סעיף 10 - MCP preview

רשות / ניתן לפרק אם חסר זמן

זהו preview לנושא שנחזור אליו בהמשך.

MCP הוא קיצור של Model Context Protocol. להיום, אתם צריכים רק את הרעיון הכללי: MCP הוא תקן שמאפשר לאפליקציות AI להתחבר לכלים ולמקורות נתונים.

זה יכול לכלול קבצים, מסדי נתונים, APIs פנימיים, מערכות תיעוד, מערכות משימות, מערכות CI, פרומפטים או תהליכי עבודה.

למה זה חשוב?

כי הקשר וכלים הופכים AI לשימושי יותר. אם העוזר יכול לראות את התיעוד הנכון, לבדוק את המאגר, לקרוא דוח בדיקות או להשתמש בכלי פנימי מבוקר, הוא יכול לעשות עבודה מועילה יותר.

אבל בדיוק אותו דבר מגדיל סיכון. אם עוזר יכול להשתמש בכלים, צריך לחשוב על:

- מה הוא יכול לקרוא;
- מה הוא יכול לכתוב;
- מי אישר את הפעולה;
- מה מתועד בלוגים;
- האם הכלי יכול לשנות מערכות אמיתיות;
- האם prompt injection יכול לגרום לשימוש לא רצוי בכלים;
- איך מגבילים או מבטלים הרשאות.

לכן מסקנת הביניים פשוטה:

ככל שהעוזר יכול לעשות יותר, כך הרשאות וביקורת חשובות יותר.

אל תדאגו אם זה עדיין מרגיש מופשט. מודול 1 רק מציג את הרעיון. בהמשך, כשנדבר על תהליכים סוכניים ואוטומציה, זה יהפוך למוחשי יותר.

אם הזמן קצר, אפשר להעביר את השקף הזה כ-teaser של שתי דקות.

שקף 16 - תרגיל כיתה: סיווג מצב AI

סמן מצגת: תרגיל סיווג

סמן מחברת: סעיף 11 - תרגיל קצר ופתרון

חובה

עכשיו נתרגל את מודל ההחלטה.

עבור כל משימה, אני רוצה שתבחרו מצב עבודה עם AI ותוכנית אימות. עבדו בזוגות או בקבוצות קטנות.

אלה המשימות:

1. להסביר מודול שמעולם לא ראיתם.
2. ליצור unit tests לפונקציה טהורה.
3. להוסיף pagination ל-API פנימי קטן.
4. לבצע refactor ללוגיקת authorization בפרודקשן.
5. לסכם PRs שמוזגו להערות גרסה.
6. ליצור סקריפט שמוחק נתוני פרודקשן ישנים.

עבור כל אחת, ענו על שלוש שאלות:

- באיזה מצב AI תשתמשו?
- איזה הקשר בטוח לספק?
- איך תאמתו את התוצאה?

אתן לכם חמש דקות. אל תנסו למצוא תשובה רשמית אחת. המטרה היא לחשוב בקול רם בעזרת חמש השאלות: בהירות, הקשר, רדיוס פגיעה, עלות אימות, והאם הכלי צריך לפעול.

[לעצור לעבודה קבוצתית].

בואו נסכם.

משימה 1: להסביר מודול שלא ראיתם. בדרך כלל אתחיל בצ'אט או מתכנת זוגי במצב קריאה בלבד. תוכנית האימות היא לבדוק מדגמית את ההסבר מול הקוד ולבקש מהעוזר להפריד בין עובדות מאומות להנחות.

משימה 2: יצירת unit tests לפונקציה טהורה. מתכנת זוגי מתאים. אבל יש לבדוק בדיקות שנוצרו. אני רוצה assertions מפורשים ומקרי קצה.

משימה 3: הוספת pagination ל-API פנימי קטן. זו יכולה להיות משימת סוכן IDE תחומה אם חוזה ה-API ברור ויש בדיקות. אבל צריך להיזהר מצורת התשובה. הוספת metadata עלולה לשבור לקוחות אם ה-endpoint החזיר בעבר רשימה.

משימה 4: refactor ללוגיקת authorization בפרודקשן. זו צריכה להיות עבודה מונחית אדם עם סיוע AI בלבד. AI יכול להסביר, לעזור לנסח בדיקות, ליצור רשימת בדיקה או לעזור בביקורת, אבל ריפקטוריןג אוטונומי רחב מסוכן מדי.

משימה 5: סיכום PRs שמוזגו להערות גרסה. צ'אט או כלי workflow יכולים להתאים. אימות משמעו השוואה לרשימת ה-PRs ועריכה לדיוק מוצרי.

משימה 6: יצירת סקריפט שמוחק נתוני פרודקשן ישנים. עבודה מונחית אדם עם AI ייעוצי בלבד. צריך מצב dry-run, גיבויים, audit logs, ביקורת עמיתים ותוכנית AI. rollback יכול לעזור לנסח checklist, אבל הפעולה עצמה מסוכנת.

הדפוס החשוב הוא ששימוש ב-AI אינו בינארי. השאלה אינה "האם AI יכול לעזור?". בדרך כלל כן. השאלה היא: "איזו צורת עבודה בטוחה, ואיזה אימות הופך אותה למקובלת?".

שקף 17 - מסקנות מרכזיות

סמן מצגת: מסקנות מודול 1

סמן מחברת: סעיף 13 - Takeaways

חובה

נסכם את מודול 1.

מסקנה ראשונה: חשבו בצורות אינטראקציה, לא בשמות מותגים.

שמות כלים ישתנו. פיצ'רים יזוזו. גבולות מוצרים ישתנו. אבל אפשר להמשיך לשאול: האם זה עוזר, צ'אט, מתכנת זוגי, סוכן IDE, סוכן קידוד או תהליך עבודה מחובר?

שנית: הגדילו אוטונומיה רק כאשר המשימה ברורה וניתנת לביקורת.

אל תשתמשו במצב אוטונומי מאוד רק משום שהוא קיים. השתמשו בו כאשר המשימה תחומה, ההקשר זמין והאימות מספיק זול.

שלישית: צפו להאצה, לא לנכונות מובטחת.

AI יכול להפוך את הטיוטה הראשונה למהירה יותר. הוא לא מבטל את הצורך באימות.

רביעית: התייחסו לקוד שנוצר כאילו הוא לא מהימן עד שנבדק.

זו לא עוינות. זו הנדסה רגילה. אנחנו בודקים גם קוד של בני אדם.

חמישית: הכניסו פרטיות, IP וממשל לתהליך העבודה מהיום הראשון.

אל תחכו לדליפה או לפעולה לא בטוחה כדי להחליט על מדיניות.

שישית: זכרו שהקשר איננו רק טקסט בפרומפט. הקשר הוא מבנה המאגר, בדיקות, תיעוד, conventions, אילוצים ולעיתים הרשאות כלים.

הגשר למודול 2 פשוט. במודול 1 בנינו את המודל המנטלי. במודול 2 נשתמש בבינה מלאכותית בתוך תהליך פיתוח רגיל: תכנון, פרומפטים, מימוש, דיבוג, בדיקות, תיעוד, ביקורת ורפלקציה.

לפני שנמשיך, אני רוצה שתצאו עם משפט אחד:

פיתוח מועצם בבינה מלאכותית אינו ויתור על הנדסה לטובת AI. הוא שימוש ברמת הסיוע הנכונה למשימה, תוך שמירה על בעלות אנושית על איכות, בטיחות ותוצאות.

זה הבסיס לכל המשך הקורס.

תסריט הדגמה אופציונלי - משימה אחת, ארבעה מצבי אינטראקציה

סמן הדגמה: `module1_demo_runbook.md`, הדגמה 1

רשות / להשתמש אם יש זמן

אם יש זמן להדגמה חיה, אפתח את `demo_repo/` ב-IDE ואציג את `src/retail_demo/profile_parser.py`.

אני אומר:

נשתמש במשימה קטנה אחת ונראה כיצד היא משתנה בין מצבי אינטראקציה. המשימה היא להוסיף אימות ל-`profile parser`. זה קטן בכוונה. הנקודה אינה ה-`parser`; הנקודה היא תהליך העבודה.

ראשית, מצב עוזר `inline`. אני מציב את הסמן בתוך הפונקציה וכותב הערה:

```
# Validate that email is present and contains '@'
```

אני נותן לעוזר להציע שינוי מקומי. ואז אני אומר:

שימו לב למה המצב הזה טוב. הוא מהיר. הוא מקומי. אבל הוא כנראה לא עדכן בדיקות, וייתכן שאינו מבין את מדיניות האימות הרחבה.

שנית, מצב הסבר בצ'אט. אני שואל:

Explain `src/retail_demo/profile_parser.py` in one paragraph. Then list the top five validation risks. Do not suggest code changes yet.

ואז אני אומר:

זהו שלב התמצאות בקריאה בלבד. לעיתים זה הצעד הראשון הבטוח ביותר בקוד לא מוכר. אם העוזר טוען טענות, נבדוק אותן מול הקוד.

שלישית, מצב תכנון של מתכנת זוגי. אני מבקש:

I want to add input validation to parse_user_profile without changing the public return type. First propose a minimal plan. Include expected behavior, affected files, tests to add, and risks. Do not edit files yet.

ואז אני אומר:

עכשיו העוזר עוזר לי לתכנן, אבל אני עדיין מוביל. אני יכול לדחות אימות אימייל קשוח מדי, להחליט על התנהגות חריגות, ולציין אילו בדיקות חשובות.

רביעית, עריכה סוכנית תחומה. אני מבקש:

Implement the smallest safe validation change for parse_user_profile. Constraints:

- Do not add third-party dependencies.
- Preserve the UserProfile dataclass and return type.
- Raise ValueError with clear messages for invalid input.
- Add focused tests for missing email, invalid email, blank name, negative age, and unsupported marketing_opt_in values.
- Keep the diff limited to profile_parser.py and test_profile_parser.py.

After editing, summarize the diff and tell me exactly what checks to run.

ואז אני מריץ:

pytest

הסיכום הוא:

אותה משימה עברה דרך כמה מצבי אינטראקציה. ככל שנתנו לכלי יותר אוטונומיה, כך מגבלות ואימות הפכו חשובים יותר.

רפלקציה אופציונלית לסיום המודול

סמן מחברת: סעיף 13 - Takeaways

רשות / שימושי כ-exit ticket

אסיים בתרגיל כתיבה קצר:

כתבו תשובה בשלושה משפטים:

1. משימה אחת שבה הייתם משתמשים ב-AI בצורה אגרסיבית.
2. משימה אחת שבה הייתם משאירים את ה-AI במצב קריאה בלבד או ייעוץ.
3. הרגל ביקורת אחד שאתם רוצים לשמור לאורך הקורס.

תשובה טובה יכולה להיות:

הייתי משתמש ב-AI בצורה אגרסיבית ל-boilerplate של בדיקות סביב פונקציות טהורות, כי ההתנהגות קלה לאימות. הייתי משאיר את ה-AI במצב קריאה בלבד עבור שינויי authorization, כי רדיוס הפגיעה גבוה וההקשר המדיניותי חשוב. אני רוצה לבקש הנחות ושלבי אימות לפני קבלת קוד שנוצר.

זה בדיוק סוג שיקול הדעת שהמודול הזה נועד לבנות.